
Neural Control: Adjoint Learning Through Equilibrium Constraints

Dezhong Tong¹ Jiawen Wang² Hengyi Zhou¹ Yinglong Shen² Xiaonan Huang¹ M. Khalid Jawed²

Abstract

Many physical AI tasks require sequential implicit computation: at each step, boundary controls are applied, and the resulting configuration is obtained by solving an equilibrium problem. This setting arises naturally in deformable object manipulation, where even bending a deformable linear object (DLO) to a target shape can be nonlinear and multistable: identical boundary conditions may produce different configurations depending on actuation history. Unlike explicit transition models, the control-to-configuration relation is implicit and history-dependent, making long-horizon learning and control brittle; backpropagating through iterative solves is also memory- and compute-intensive. We propose Neural Control, a boundary-control framework that propagates gradients through branch-dependent sequences of equilibrium solves rather than a single fixed point. Neural Control computes trajectory-dependent proxy gradients by differentiating equilibrium conditions with an adjoint formulation, avoiding solver unrolling while keeping forward rollouts on converged equilibria. Combined with receding-horizon continuation, Neural Control re-anchors optimization to realized equilibria and mitigates basin switching. We validate Neural Control on simulated and real DLO manipulation, compare against SPSA and iCEM, and demonstrate applicability to a learned DEQ-style implicit equilibrium model.

1. Introduction

Many learning and control problems in physical AI require optimizing through sequential implicit computation: instead

¹The Robotics Department, University of Michigan, Ann Arbor, US ²The Department of Mechanical and Aerospace Engineering, University of California, Los Angeles, US. Correspondence to: Xiaonan Huang <xiaonanh@umich.edu>, M. Khalid Jawed <khalidjm@seas.ucla.edu>.

of producing the next state with an explicit transition map, each state is obtained by solving an implicit equilibrium or fixed-point problem. This setting arises naturally in physical AI, robotics, graphics, and simulation, where configurations are often defined as minimizers of an energy functional under boundary conditions, loads, or design parameters. In such systems, control changes the implicit problem itself: by adjusting boundary conditions or parameters, the agent shapes the energy landscape so that the resulting equilibrium satisfies a desired objective. The resulting control-to-configuration relation is therefore solver-defined rather than explicit, and it can be high-dimensional, nonlinear, and multistable.

This sequential implicit setting poses challenges beyond differentiating a single fixed point. In multistable systems, the same control input can admit multiple valid equilibria, and the realized state is selected by the previous configuration, actuation history, and basin of attraction of the warm-started solver. Thus, the rollout is branch-dependent and history-dependent: small changes in actuation can move the system to a different equilibrium branch. This challenges the single-valued transition assumption underlying many model-learning and planning methods, and makes long-horizon credit assignment brittle (Hausknecht & Stone, 2015; Suh et al., 2022).

A representative example is robotic manipulation of deformable structures (Figure 1A), such as deformable linear objects (DLOs) (Mora et al., 2021; Li et al., 2023; Tong et al., 2024). Even seemingly simple tasks, such as bending an elastic strip to a target shape, involve large geometrically nonlinear deformation, hysteresis, and multistable equilibria (Qiao et al., 2020; Tong et al., 2025; Huang et al., 2025; Bretl & McCarthy, 2014; Tong et al., 2021). As illustrated in Figure 1B, identical boundary conditions can correspond to multiple stable configurations. DLO manipulation therefore provides a challenging testbed for sequential implicit optimization: each rollout step requires solving an equilibrium problem over a high-dimensional continuum discretization (Yin et al., 2021; Moll & Kavraki, 2006; Bergou et al., 2008), and the realized branch depends on the actuation path.

Differentiable simulation provides gradients for physical optimization problems (Qiao et al., 2020; Mora et al., 2021;

Suh et al., 2022), but long-horizon implicit rollouts introduce two additional difficulties. First, multistable control requires branch-consistent, trajectory-dependent sensitivities that follow the realized branch selected by the solver (Georgiev et al., 2024; Gao et al., 2024). If the branch structure is ignored, learned dynamics may blur or average over multiple outcomes; latent-state or multimodal models can partially mitigate this issue (Bishop, 1994; Hafner et al., 2019b;a). Second, naively backpropagating through every Newton or conjugate-gradient iteration is memory-intensive and can be numerically fragile near instabilities (Gruslys et al., 2016; Amos & Kolter, 2017; Bai et al., 2019).

Existing implicit-layer methods typically differentiate a single solver-defined object, such as one optimization layer or KKT system (Amos & Kolter, 2017; Agrawal et al., 2019; Amos et al., 2018), or one fixed point in a deep equilibrium model (Bai et al., 2019). In contrast, our setting requires propagating useful gradients through a sequence of warm-started, branch-dependent equilibrium solves. The central challenge is therefore not only to differentiate an implicit state, but to obtain trajectory-level sensitivities that remain aligned with the realized equilibrium branch.

We propose Neural Control¹, a boundary-control framework for long-horizon optimization through sequential implicit equilibria. The forward rollout is solver-consistent: each state is obtained by solving the equilibrium equations to convergence along the realized branch. Inspired by adjoint sensitivity methods in continuous-depth learning (Chen et al., 2018), we differentiate the equilibrium conditions via an adjoint formulation to compute trajectory-dependent proxy gradients, avoiding unrolling of the inner Newton/CG iterations. This provides a memory-efficient gradient pathway for optimizing control signals through sequences of implicit solves.

To improve robustness in multistable regimes, we combine these gradients with receding-horizon continuation (RHC), related in spirit to receding-horizon model predictive control (Chua et al., 2018). In this setting, RHC serves as a continuation and branch-tracking mechanism: by repeatedly re-anchoring optimization to realized equilibria, it localizes proxy-gradient error, reduces basin drift, and mitigates unintended switches between equilibrium branches.

We validate Neural Control on shape and trajectory control of DLOs using both simulation and real-robot experiments under noise, model mismatch, and gravity. We compare against black-box gradient-free baselines, including SPSA (Spall, 2002) and iCEM (Pinneri et al., 2021), with CEM (Botev et al., 2013) reported in the appendix as an additional classical baseline, and show that Neural Control

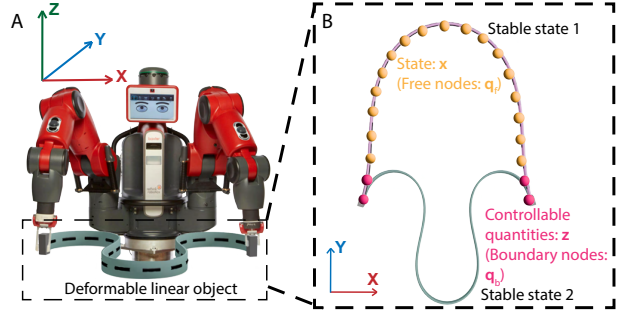


Figure 1. (A) Robotic manipulation of a deformable linear object (e.g., elastic strips). (B) Discretization into boundary and free nodes. Varying the controllable quantities $\mathbf{z}$ shapes the resulting state $\mathbf{x}$. Multiple stable shapes may exist under the same $\mathbf{z}$.

achieves substantially lower loss with significantly less wall-clock time. To further demonstrate that the framework can operate beyond analytical mechanics models, we also evaluate it on a learned DEQ-style implicit equilibrium model trained from experimental force–strain data.

This paper makes the following contributions: (i) a sequential implicit optimization formulation for branch-dependent equilibrium rollouts, clarifying the distinction from single-step implicit layers and fixed-point models; (ii) a proxy-adjoint gradient method that computes trajectory-dependent sensitivities through converged equilibrium solves without unrolling inner Newton/CG iterations; (iii) a receding-horizon continuation scheme that re-anchors optimization to realized equilibria and improves branch consistency in multistable regimes; (iv) empirical validation on simulated and real DLO manipulation, comparison with modern gradient-free baselines, including iCEM, and an additional learned DEQ-style implicit-model experiment.

2. Related Work

Optimization and Learning for Long-Horizon, Path-Dependent Control. Long-horizon control and sequential decision-making are central themes in machine learning, where performance is often limited by delayed credit assignment, partial observability, and, in model-based settings, compounding model error. A common response is to treat history dependence as partial observability and augment policies or value functions with memory, e.g., recurrent reinforcement learning (RL) (Hausknecht & Stone, 2015), or to learn latent state-space models that support planning through imagined rollouts (Hafner et al., 2019b;a). Another dominant paradigm is model-based RL with receding-horizon planning: actions are optimized over a short horizon and repeatedly replanned using model predictive control (MPC), improving robustness to model error and regime changes (Chua et al., 2018).

¹Code, videos, and additional results are available at <https://structurescomp.github.io/neural-control/>.

Black-box and sampling-based optimizers are also widely used for long-horizon problems because they avoid differentiating through complex transitions and can explore multi-modal outcomes. Representative methods include simultaneous perturbation stochastic approximation (SPSA) (Spall, 2002), the cross-entropy method (CEM) (Botev et al., 2013), sample-efficient variants such as iCEM (Pinneri et al., 2021), and parameter-space search methods such as evolution strategies (ES) (Salimans et al., 2017) and augmented random search (ARS) (Mania et al., 2018). Related work also makes CEM differentiable (DCEM) (Amos & Yarats, 2020). However, these methods typically require many full rollouts per update, and their cost grows quickly with horizon length, action dimension, and population size. This cost becomes especially acute in our setting, where each rollout step itself requires an expensive implicit equilibrium solve. Our method retains the stabilizing effect of receding-horizon replanning, but computes trajectory-level sensitivities through the realized implicit rollout, enabling efficient gradient-based optimization.

Memory-Efficient Gradients Through Implicit Computation. Backpropagation through time (BPTT) provides a direct way to obtain trajectory-dependent gradients by propagating sensitivities across timesteps (Williams & Peng, 1990), but storing intermediate states makes memory scale with horizon length, motivating checkpointing and recomputation strategies (Gruslys et al., 2016). In differentiable physics and control, this issue is amplified when each transition is computed by an iterative implicit solver, such as Newton or conjugate gradient, since differentiating through all solver iterations is memory-intensive and can be numerically brittle near instabilities (Amos & Kolter, 2017; Suh et al., 2022).

A complementary line of work avoids solver unrolling by differentiating through optimality, fixed-point, or KKT conditions. Examples include differentiable optimization layers such as OptNet (Amos & Kolter, 2017) and differentiable convex optimization layers (Agrawal et al., 2019), deep equilibrium models (DEQs) (Bai et al., 2019), differentiable MPC via KKT conditions (Amos et al., 2018), and Pontryagin differentiable programming (PDP) (Jin et al., 2020). Inspired by adjoint sensitivity methods for continuous-depth models (Chen et al., 2018), these methods show that gradients can often be obtained without unrolling the full computational procedure.

Our setting differs from these methods in that the object being differentiated is not a single optimization layer, fixed point, or KKT system. Instead, we require gradients through a sequence of warm-started equilibrium solves, where multistability makes the realized trajectory branch-dependent. Compared to BPTT through solver iterations, Neural Control avoids unrolling inner solves; compared to standard

implicit layers, it propagates proxy-adjoint sensitivities through long-horizon, branch-dependent implicit rollouts.

Differentiable Models for Physical Learning and Control. Differentiable modeling and simulation have become core primitives for gradient-based learning, planning, and inverse problems. ML toolchains such as DiffTaichi (Hu et al., 2019a) and Brax (Freeman et al., 2021) make optimization through physics practical at scale. Differentiable simulators have also been extended from rigid bodies to deformable and soft systems, including ChainQueen (Hu et al., 2019b), DiffPD (Du et al., 2021), and differentiable soft multi-body simulation (Qiao et al., 2021). For flexible structures, discrete differential geometry (DDG) discretizations provide geometry-preserving, energy-based models with analytic derivatives (Bergou et al., 2008; Grinspun et al., 2003; Huang et al., 2025; Tong et al., 2025). Benchmarks such as SoftGym (Lin et al., 2021) and PlasticineLab (Huang et al., 2021) further highlight the difficulty of long-horizon deformable manipulation.

Learned differentiable simulators and graph-based dynamics models can provide fast predictors for physical systems (Sanchez-Gonzalez et al., 2020; Pfaff et al., 2021), while analytic gradients from differentiable multiphysics simulations have also been integrated into RL updates, e.g., SAPO (Xing et al., 2025). However, near multistability and bifurcation boundaries, deterministic predictors can blur branch selection unless multimodality or latent modes are modeled explicitly (Bishop, 1994). Our work is complementary to these modeling efforts: rather than proposing a new simulator, we focus on the gradient pathway for optimizing through sequential implicit models. This perspective applies both to analytical mechanics models and to learned implicit equilibrium models, as demonstrated by our DEQ-style validation.

3. Methodology

We study learning and control problems where the system state is defined implicitly by an equilibrium condition rather than an explicit dynamics model. A continuation parameter $\lambda \in [0, 1]$ indexes the process, with $\lambda = 0$ the initial configuration and $\lambda = 1$ the terminal configuration. At each λ , the equilibrium state $\mathbf{x}(\lambda)$ is obtained by solving

$$\mathcal{G}(\mathbf{x}(\lambda), \mathbf{z}(\lambda)) = \mathbf{0}, \quad (1)$$

where $\mathbf{z}(\lambda)$ denotes controllable quantities entering the equilibrium problem, such as boundary conditions, loads, or design parameters. Thus, the state is not produced by an explicit transition map of the form $\mathbf{x}_{k+1} = \mathcal{F}(\mathbf{x}_k, \mathbf{z}_k)$, but by a solver-selected equilibrium. In mechanics, Eq. (1) represents quasi-static force balance: inertia and vibrational effects are neglected, so internal conservative forces balance

applied loads. A common instance is

$$\mathcal{G}(\mathbf{x}, \mathbf{z}) = \nabla_{\mathbf{x}} E(\mathbf{x}, \mathbf{z}) - \mathbf{F}^{\text{ext}}, \quad (2)$$

where E is total potential energy and $\mathbf{F}^{\text{ext}}$ collects external loads.

Equilibrium-governed systems pose two challenges: (i) $\mathbf{x}(\lambda)$ is typically computed by iterative solvers (e.g., Newton–CG), so differentiating through solver iterations across long horizons is expensive; and (ii) equilibria can be multistable, so for fixed $\mathbf{z}$, multiple stable solutions may exist and warm-started solvers follow a realized branch through their basins of attraction. We therefore generate rollouts by warm-started equilibrium solves along the continuation path. In all experiments, each forward state is solved to the prescribed equilibrium tolerance, so the backward approximation does not arise from truncating or unrolling the forward solver.

A neural controller parameterized by Θ produces a continuation control signal

$$\mathbf{u}(\lambda) = \mathbf{u}_{\Theta}(\lambda), \quad (3)$$

which drives $\mathbf{z}(\lambda)$ via known continuation dynamics

$$\frac{d\mathbf{z}}{d\lambda} = f(\mathbf{z}(\lambda), \mathbf{u}(\lambda)), \quad \mathbf{z}(0) = \mathbf{z}_0. \quad (4)$$

Given $\mathbf{z}(\lambda)$, the realized equilibrium $\mathbf{x}(\lambda)$ is obtained by repeatedly solving Eq. (1) with warm starts from the previous solution, yielding a branch-consistent rollout.

3.1. Adjoint Gradients via Proxy Sensitivity Dynamics

At any converged equilibrium $(\mathbf{x}(\lambda), \mathbf{z}(\lambda))$ satisfying $\mathcal{G}(\mathbf{x}, \mathbf{z}) = \mathbf{0}$, and assuming the realized branch is locally smooth with nonsingular $\mathbf{G}_x$, implicit differentiation gives the local equilibrium sensitivity

$$\mathbf{S}(\lambda) := \frac{\partial \mathbf{x}}{\partial \mathbf{z}}(\lambda), \quad \mathbf{G}_x(\lambda) \mathbf{S}(\lambda) = -\mathbf{G}_z(\lambda), \quad (5)$$

where $\mathbf{G}_x = \partial \mathcal{G} / \partial \mathbf{x}$ and $\mathbf{G}_z = \partial \mathcal{G} / \partial \mathbf{z}$. Near bifurcations or ill-conditioned equilibria, $\mathbf{G}_x$ may become singular or poorly conditioned, and the local sensitivity can become unreliable. We never form $\mathbf{S}(\lambda)$ explicitly. To compute a product $\mathbf{S}(\lambda)^\top \mathbf{v}$, we solve the adjoint linear system

$$\mathbf{G}_x(\lambda)^\top \boldsymbol{\eta} = \mathbf{v} \quad (6)$$

and return

$$\mathbf{S}(\lambda)^\top \mathbf{v} = -\mathbf{G}_z(\lambda)^\top \boldsymbol{\eta}. \quad (7)$$

This requires one adjoint linear solve and avoids constructing the dense sensitivity matrix.

Along a smooth realized branch, the total derivative satisfies

$$\frac{d\mathbf{x}}{d\lambda} = \mathbf{S}(\lambda) \frac{d\mathbf{z}}{d\lambda} + \frac{\partial \mathbf{x}}{\partial \lambda}. \quad (8)$$

In our instantiation, $\mathcal{G}$ has no explicit λ -dependence: any scheduling is absorbed into $\mathbf{z}(\lambda)$. Consequently, $\partial \mathbf{x} / \partial \lambda = \mathbf{0}$ and

$$\frac{d\mathbf{x}}{d\lambda} = \mathbf{S}(\lambda) \frac{d\mathbf{z}}{d\lambda}. \quad (9)$$

Combining Eq. (9) with the continuation dynamics Eq. (4) gives the continuous sensitivity identity

$$\frac{d\mathbf{x}}{d\lambda} = \mathbf{S}(\lambda) f(\mathbf{z}(\lambda), \mathbf{u}(\lambda)). \quad (10)$$

Eq. (10) defines the proxy sensitivity dynamics used for backward sensitivity propagation. The forward states $\mathbf{x}(\lambda)$ are still obtained by solving Eq. (1) to convergence at each continuation step. Thus, the forward rollout remains solver-consistent, while the backward pass uses local implicit sensitivities evaluated along the realized branch.

The main analytic approximation in our gradient construction is a frozen-tangent discretization. We evaluate $\mathbf{S}(\lambda)$ on the realized rollout at discrete continuation steps and treat it as locally constant during the backward pass, thereby ignoring derivatives of $\mathbf{S}$ with respect to $\mathbf{x}$, $\mathbf{z}$, and λ along the segment. This avoids higher-order derivatives of Eq. (1). The approximation is local: it assumes that the realized equilibrium branch is smooth and that $\mathbf{G}_x$ is nonsingular and varies moderately over each continuation step. Near bifurcations, ill-conditioned equilibria, or high-curvature branches, the local tangent can change rapidly and the proxy gradient may become less accurate. Appendix C formalizes this approximation and shows that, under these local regularity assumptions, the resulting proxy gradient is first-order accurate.

Terminal Objective. We first consider a terminal loss

$$\mathcal{L} = \phi(\mathbf{x}(1), \mathbf{z}(1)). \quad (11)$$

Define proxy-system costates $\mathbf{a}(\lambda) := \partial \mathcal{L} / \partial \mathbf{x}(\lambda)$ and $\mathbf{g}(\lambda) := \partial \mathcal{L} / \partial \mathbf{z}(\lambda)$. Under Eq. (10) and frozen-tangent approximation, the terminal conditions are

$$\begin{aligned} \mathbf{a}(1) &= \nabla_{\mathbf{x}} \phi(\mathbf{x}(1), \mathbf{z}(1)), \\ \mathbf{g}(1) &= \nabla_{\mathbf{z}} \phi(\mathbf{x}(1), \mathbf{z}(1)). \end{aligned} \quad (12)$$

Let $\mathbf{A}(\lambda) := \partial f / \partial \mathbf{z}(\mathbf{z}(\lambda), \mathbf{u}(\lambda))$ and $\mathbf{B}(\lambda) := \partial f / \partial \mathbf{u}(\mathbf{z}(\lambda), \mathbf{u}(\lambda))$. Under frozen tangent, we neglect derivatives of $\mathbf{S}(\lambda)$ along the branch. The costates satisfy the backward ODEs, integrated from $\lambda = 1$ to 0,

$$\begin{aligned} -\frac{d\mathbf{a}}{d\lambda} &= \mathbf{0}, \\ -\frac{d\mathbf{g}}{d\lambda} &= \mathbf{A}(\lambda)^\top (\mathbf{g}(\lambda) + \mathbf{S}(\lambda)^\top \mathbf{a}(\lambda)), \end{aligned} \quad (13)$$

and the instantaneous control gradient is

$$\frac{d\mathcal{L}}{d\mathbf{u}(\lambda)} = \mathbf{B}(\lambda)^\top (\mathbf{g}(\lambda) + \mathbf{S}(\lambda)^\top \mathbf{a}(\lambda)). \quad (14)$$

The parameter gradient follows by the chain rule

$$\frac{d\mathcal{L}}{d\Theta} = \int_0^1 \left(\frac{\partial \mathbf{u}_\Theta(\lambda)}{\partial \Theta} \right)^\top \frac{d\mathcal{L}}{d\mathbf{u}(\lambda)} d\lambda. \quad (15)$$

3.2. Adjoints for Trajectory Objectives

Many equilibrium-control tasks specify objectives along the realized branch rather than only at $\lambda = 1$. We therefore consider

$$\mathcal{L} = \phi(\mathbf{x}(1), \mathbf{z}(1)) + \int_0^1 w(\lambda) \ell(\mathbf{x}(\lambda), \mathbf{z}(\lambda)) d\lambda, \quad (16)$$

where $w(\lambda) \geq 0$ is user-specified. The terminal conditions remain Eq. (12). Under Eq. (10) and frozen tangent, the backward ODEs become

$$\begin{aligned} -\frac{d\mathbf{a}}{d\lambda} &= w(\lambda) \nabla_{\mathbf{x}} \ell(\mathbf{x}(\lambda), \mathbf{z}(\lambda)), \\ -\frac{d\mathbf{g}}{d\lambda} &= w(\lambda) \nabla_{\mathbf{z}} \ell(\mathbf{x}(\lambda), \mathbf{z}(\lambda)) \\ &\quad + \mathbf{A}(\lambda)^\top (\mathbf{g}(\lambda) + \mathbf{S}(\lambda)^\top \mathbf{a}(\lambda)). \end{aligned} \quad (17)$$

The control and parameter gradients retain the same form as Eqs. (14)–(15).

3.3. Receding-Horizon Continuation Control

Over long continuations, proxy-gradient and discretization errors can accumulate. In multistable regions, these errors may push the warm-started equilibrium solve across basins of attraction, causing the rollout to switch to an unintended equilibrium branch. We therefore solve the task as a sequence of short continuation segments, in the spirit of receding-horizon optimization. In our setting, receding-horizon continuation (RHC) serves as a continuation and branch-tracking mechanism: it repeatedly re-anchors optimization to the realized equilibrium branch.

At segment index m , starting from $(\mathbf{x}_0^{(m)}, \mathbf{z}_0^{(m)})$, we optimize a control schedule over a local homotopy $\lambda \in [0, 1]$, implemented with H continuation steps of size $1/H$. The continuation control is generated by a lightweight neural controller

$$\mathbf{u}(\lambda) = \mathbf{u}_{\Theta^{(m)}}(\lambda), \quad (18)$$

with fixed architecture and parameters $\Theta^{(m)}$ reinitialized at the start of each segment and optimized within the segment. We minimize a short-horizon segment objective $\mathcal{L}^{(m)}$ of the same form as Eq. (16), restricted to the segment rollout:

$$\begin{aligned} \min_{\Theta^{(m)}} \mathcal{L}^{(m)} &= \phi(\mathbf{x}^{(m)}(1), \mathbf{z}^{(m)}(1)) \\ &\quad + \int_0^1 w(\lambda) \ell(\mathbf{x}^{(m)}(\lambda), \mathbf{z}^{(m)}(\lambda)) d\lambda. \end{aligned} \quad (19)$$

subject to the equilibrium constraint (1), continuation dynamics (4), and bounded progress (e.g., $\|\mathbf{u}(\lambda)\| \leq u_{\max}$).

We optimize $\Theta^{(m)}$ using the proxy-adjoint gradients above and stop early when the segment objective stalls, or after a fixed budget. Executing the optimized segment yields $(\mathbf{x}^{(m)}(1), \mathbf{z}^{(m)}(1))$, and we shift the horizon by

$$\mathbf{x}_0^{(m+1)} \leftarrow \mathbf{x}^{(m)}(1), \quad \mathbf{z}_0^{(m+1)} \leftarrow \mathbf{z}^{(m)}(1). \quad (20)$$

This segment-wise design localizes proxy-gradient error, reduces basin drift, and improves branch consistency in multistable landscapes. We provide the discrete derivation, implementation details, and pseudocode in Appendix A and Algorithm 1.

4. Results

We evaluate Neural Control on elastic-strip manipulation tasks (Figure 1A). The strip undergoes large deformations and admits multistable equilibria, inducing strong path dependence: small continuation errors can accumulate and trigger undesired switches between equilibrium branches. We consider three tasks that progressively stress these effects: (i) point-to-point manipulation with a terminal objective, (ii) path following of a designated strip point with a trajectory objective, and (iii) shape formation in multistable regimes requiring branch selection.

All methods share the same forward equilibrium solver, which computes strip configurations by solving implicit equilibria to convergence at each continuation step. We compare our full method (Adjoint+RHC) against gradient-free baselines, including SPSA and iCEM, and ablate our approach by removing RHC (Adjoint-only). CEM results are provided in Appendix E as an additional classical gradient-free baseline.

Each method is run under a fixed update budget. We report both best achieved loss and total wall-clock time, including all equilibrium solves and additional tangent/adjoint linear-solve overheads. Finally, we validate the approach in real-world robotic experiments; an additional learned DEQ-style implicit-model experiment is provided in Appendix F.

4.1. Experimental Setup

We model the manipulated object as a planar elastic strip with stretching and bending energy. At each continuation step, the forward rollout computes the realized configuration by solving the corresponding equilibrium equations to convergence under imposed boundary constraints. We partition degrees of freedom into boundary nodes $\mathbf{q}_b$ and free nodes $\mathbf{q}_f$ and set $\mathbf{z} \equiv \mathbf{q}_b$, $\mathbf{x} \equiv \mathbf{q}_f$ (Figure 1B): control acts only through boundary conditions, while free nodes are solved implicitly and used to evaluate task losses. The strip is held by clamped boundary conditions at both ends; Tasks 1–2

actuate transverse (global x -axis in our setup) displacements at both ends, and Task 3 actuates (x, y) displacement and in-plane rotation at one end, with the other end passively clamped. Unless otherwise stated, we discretize the strip with $N + 1 = 101$ nodes and ignore gravity in numerical experiments. The stretching and bending stiffness are fixed to $k_s = 0.314$ N and $k_b = 7.85 \times 10^{-8}$ N m². Full discretization and solver details are provided in the Appendix B.

4.2. Tasks and Metrics

We evaluate three numerical tasks that progressively test terminal objectives, trajectory objectives, and multistable shape/branch selection. In all tasks, boundary degrees of freedom define the controllable variable $\mathbf{z} \equiv \mathbf{q}_b$, and the free degrees of freedom $\mathbf{x} \equiv \mathbf{q}_f$ are solved implicitly from equilibrium and used to evaluate losses. Tasks 1–2 start from the same pre-deformed configuration (Figure 2A), while Task 3 starts from a straight strip (Figure 3).

The continuation control is produced by a neural controller $\mathbf{u}(\lambda) = \mathbf{u}_\Theta(\lambda)$, where $\mathbf{u}(\lambda)$ specifies the rate of boundary actuation along continuation. In Tasks 1–2, $\mathbf{u}(\lambda) \in \mathbb{R}^2$ outputs the transverse (x -direction) displacement rates of the left and right clamped ends. In Task 3, $\mathbf{u}(\lambda) \in \mathbb{R}^3$ outputs the (x, y) displacement rates and in-plane rotation rate of the manipulated end, while the other end remains passively clamped.

Task 1: Point-to-Point Node Targeting. We select a strip node i and a target position $\mathbf{p}_i^*$ within the reachable workspace. The goal is to drive the node to the target at the end of continuation. We use the terminal loss

$$\mathcal{L}_{\text{pos}} = \frac{1}{2} \|\mathbf{p}_i(1) - \mathbf{p}_i^*\|_2^2, \quad (21)$$

and declare success if $\mathcal{L}_{\text{pos}} \leq \varepsilon_{\text{pos}}$.

Task 2: Trajectory Tracking. We track a prescribed planar path with a designated node, chosen as the middle node. Let $\mathbf{p}_{\text{mid}}^*(\lambda)$ denote the reference trajectory, e.g., sinusoidal, circular, square-wave, or triangular paths (Figure 2B). We define

$$\ell_{\text{traj}}(\lambda) = \frac{1}{2} \|\mathbf{p}_{\text{mid}}(\lambda) - \mathbf{p}_{\text{mid}}^*(\lambda)\|_2^2, \quad (22)$$

and evaluate the trajectory objective

$$\mathcal{L}_{\text{traj}} = \int_0^1 \ell_{\text{traj}}(\lambda) d\lambda, \quad (23)$$

implemented as a Riemann sum over realized continuation steps and, for RHC, over the concatenated executed trajectory. We declare success if $\mathcal{L}_{\text{traj}} \leq \varepsilon_{\text{traj}}$.

Task 3: Shape Formation Under Multistability. We shape the strip by manipulating one clamped end while the other

end remains passively clamped. For fixed boundary conditions, multiple stable equilibria may coexist; given a target shape, the goal is to reach the corresponding equilibrium branch. We measure shape discrepancy by curvature matching:

$$\mathcal{L}_\kappa = \int_0^L (\kappa(s; 1) - \kappa^*(s))^2 ds, \quad (24)$$

where $\kappa^*(s)$ is the target curvature. We declare success if $\mathcal{L}_\kappa \leq \varepsilon_\kappa$.

4.3. Main Numerical Results

We report the main numerical results of Neural Control on the three tasks in Section 4.2. Unless otherwise noted, all experiments use the same strip discretization and physical parameters as in Section 4.1, with solver and hyperparameter details deferred to the Appendix D.

Tasks 1–2: Two-End Boundary Actuation. Figure 2 summarizes four representative instances of point-to-point node targeting (Task 1; Figure 2A) and midpoint trajectory tracking (Task 2; Figure 2B). In Task 1, the controller steers a user-selected node to the target position by coordinating the transverse actuation of both clamped ends, producing coupled rigid motion and elastic deformation along the realized equilibrium branch. In Task 2, the controller guides the strip midpoint to follow prescribed nontrivial planar trajectories; the tracked paths closely match the references across all cases, confirming that the method can optimize objectives that depend on intermediate equilibria rather than only the terminal state.

Task 3: Multistable Shape Formation. Figure 3 visualizes shape formation under multistability when manipulating a single clamped end while the other end remains passively clamped. For the same boundary condition, multiple stable equilibria coexist, including mirror-symmetric C-shaped solutions (Figure 3A1–A2) and distinct U/M-like shapes (Figure 3B1–B2). Despite these competing basins, curvature-based shaping drives the strip to the specified target configuration, indicating robust branch selection in regimes where small continuation errors can otherwise induce unintended basin switches.

4.4. Baselines and Ablations

We compare four optimizers for elastic-strip manipulation: (i) SPSA, (ii) iCEM, (iii) our neural continuation control without RHC (Adjoint-only; full-horizon), and (iv) our full method, which combines the same adjoint-based neural control with RHC (Adjoint+RHC). All methods share the same forward simulator (implicit equilibrium solves to convergence at each continuation step), and the same controller parameterization $\mathbf{u}_\Theta(\lambda)$ with λ as the sole input. They differ only in how the controller parameters Θ are updated

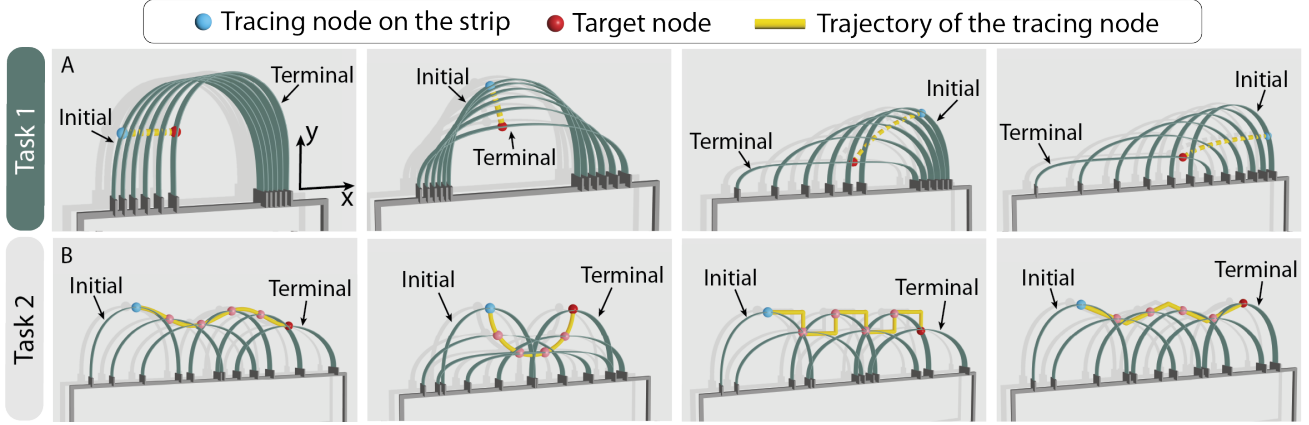


Figure 2. Numerical results for Tasks 1–2. (A) Four representative cases of two-end boundary actuation that drive a selected strip node to a user-specified target position. (B) Four representative cases of two-end boundary actuation that guide the strip midpoint along a prescribed planar trajectory. (Shadow is rendered for visualization only.)

and whether receding-horizon execution is used. CEM results are reported in Appendix E as an additional classical gradient-free baseline.

Budgeting and Evaluation. Each method is run under a fixed update budget. An objective evaluation corresponds to one full rollout that computes the task loss from implicit equilibrium solves. The number of objective evaluations per update is method-dependent: SPSA uses two evaluations per update, iCEM evaluates a population of sampled candidates, and Adjoint-only / Adjoint+RHC use one rollout per update. Adjoint-based methods additionally solve tangent/adjoint linear systems to obtain gradients via implicit sensitivities, without backpropagating through the equilibrium solver iterations. For RHC, an update refers to one optimizer update within the current segment, and all reported losses are computed on the concatenated executed trajectory across segments. We report the best loss achieved within the update

budget and the total wall-clock time in Table 1; wall-clock time includes all equilibrium solves and additional linear-solve overheads.

Baselines and Ablation. SPSA and iCEM are used as black-box optimizers over the controller parameters Θ and are run without RHC. SPSA forms a stochastic descent direction using two-sided random perturbations of Θ . iCEM samples a population of candidate controller parameters, selects elite candidates, smooths the population update, and reinserts the current best candidate. In our implementation, we use population size $P = 10$, elite fraction 0.3, smoothing coefficient $\alpha = 0.25$, minimum standard deviation 10^{-2} , elite reuse fraction 1.0, no population decay, and best-candidate reinsertion. Additional hyperparameters are provided in Appendix D. Our ablation isolates the contribution of RHC by comparing Adjoint-only against Adjoint+RHC. In Adjoint+RHC, we split the continuation into short segments of H steps and solve a sequence of segment subproblems; the controller parameters Θ are reinitialized at the start of each segment. All reported task losses are evaluated using the task definitions in Section 4.2 on the concatenated executed rollout across segments.

Convergence and Robustness. Figure 4 plots the log best-so-far objective value versus optimizer updates for all methods and tasks. Task 1 (node targeting) is the simplest setting: all methods reduce the loss early, but the gradient-free baselines plateau under the fixed update budget. Both adjoint-based variants continue improving and reach lower final loss, with Adjoint-only converging fastest in this short-horizon regime where proxy-adjoint gradients remain accurate.

Task 2 (trajectory tracking) is substantially harder because the objective depends on intermediate equilibria along the realized branch. SPSA and iCEM again stall under the fixed update budget. Adjoint-only succeeds on smooth references

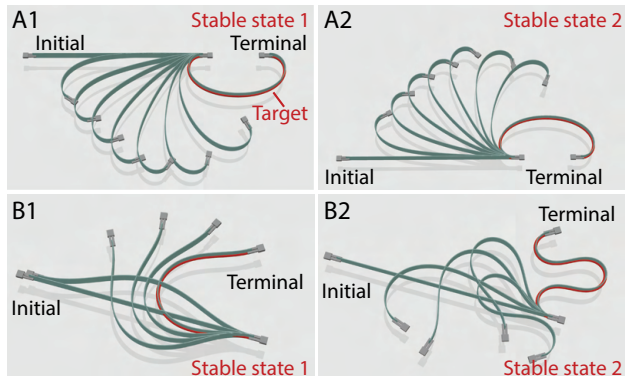


Figure 3. Neural continuation control for multistable shape formation (Task 3). (A1–A2) Two mirrored C-shaped stable equilibria arise under the same boundary condition (identical end pose), illustrating multistability. (B1–B2) A U-shaped and an M-like stable equilibrium are also obtained under the same boundary conditions.

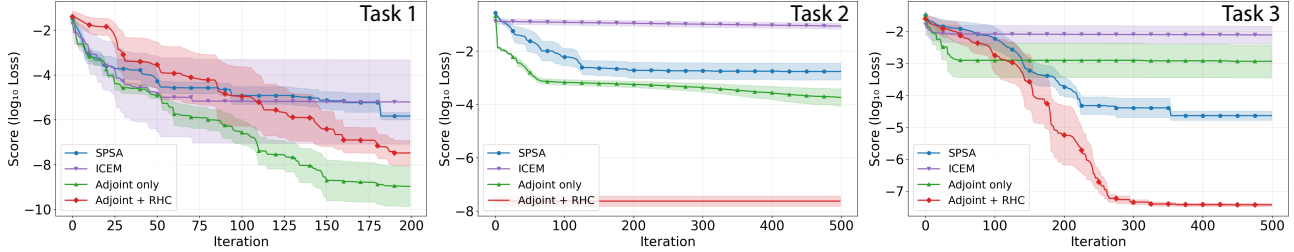


Figure 4. Optimization steps (iterations) vs. score (log best-so-far loss) for Tasks 1–3 (mean ± 1 stderr over four cases). For Task 2, Adjoint+RHC executes the trajectory in segments; we therefore report its full-horizon trajectory objective $\mathcal{L}_{\text{traj}}$ only when a complete rollout is available (i.e., at the end of the executed horizon), and use those values to form the best-so-far curve.

but struggles on non-smooth targets, where long-horizon accumulation makes optimization brittle. In contrast, Adjoint+RHC achieves consistent convergence across all trajectory types, indicating that segment-wise re-anchoring to realized equilibria is critical for stable trajectory-level control.

Task 3 (multistable shape formation) is the most challenging regime: multiple stable equilibria coexist under identical boundary conditions, so small errors can induce undesired basin switches. SPSA and iCEM stagnate early, and Adjoint-only, despite a fast initial decrease, often becomes trapped in an incorrect branch and fails the curvature tolerance. Adjoint+RHC consistently reaches the lowest curvature losses and reliably selects the specified equilibrium branch, demonstrating the importance of combining adjoint-based gradients with RHC in multistable landscapes.

Efficiency Summary. Table 1 reports best loss and wall-clock time (mean $\pm$ std) under the fixed update budget. Adjoint+RHC is consistently the most efficient method, achieving substantially lower loss with lower runtime than SPSA and iCEM. This gap is expected: gradient-free methods spend most compute on repeated full equilibrium-defined rollouts, whereas adjoint-based updates extract a search direction from one realized rollout plus additional tangent/adjoint linear solves. Compared to Adjoint-only, RHC mitigates long-horizon error accumulation and reduces trajectory-storage memory by backpropagating only through short segments, yielding the best time–accuracy trade-off in our experiments.

4.5. Real-World Experiments

As shown in Figure 5, a 6-DoF robot arm actuates one endpoint of an elastic strip, while the other endpoint is fixed using a passive clamp. The strip has measured length ≈ 25 cm, width 0.5 cm, and thickness 0.127 mm. Clamping one end yields an effective free length of 22 cm for Tasks 1–2 and 15 cm for Task 3, chosen to fit within the robot workspace. At each step, the policy outputs a desired boundary increment for the actuated endpoint, which we convert into a

Cartesian end-effector command. For Tasks 1–2, the end-effector is constrained to translate in the x – y plane; for Task 3, we additionally command an endpoint rotation. Figure 5 shows representative hardware rollouts. Across all three tasks, the observed strip motion matches the task requirements, indicating qualitative sim-to-real transfer under gravity and experimentally identified material parameters. Additional intermediate frames and videos are available on the project page¹.

5. Discussion

Our results highlight two recurring difficulties in long-horizon optimization through sequential implicit computation. First, each state is defined by a converged equilibrium solve rather than an explicit transition map, making direct backpropagation through all Newton–CG iterations memory- and compute-intensive. Neural Control addresses this by keeping the forward rollout solver-consistent, with each state solved to the prescribed equilibrium tolerance, while computing proxy-adjoint gradients from local implicit sensitivities. Second, multistable equilibrium systems are branch-dependent: small actuation changes or accumulated gradient errors can move a warm-started solver to an unintended basin of attraction. RHC mitigates this issue by acting as a continuation and branch-tracking mechanism, repeatedly re-anchoring optimization to realized equilibria and localizing proxy-gradient errors to short segments.

Although our main experiments focus on elastic-strip manipulation, the framework applies more broadly to differentiable implicit models whose states are obtained from converged solves and whose local sensitivities can be computed through the equilibrium residual. This includes analytical mechanics models, differentiable equilibrium simulators, and learned implicit equilibrium models, as illustrated by our DEQ-style validation.

The method has several limitations. It assumes a quasi-static regime and does not model inertia or fast transient dynamics. The derivation also assumes a locally smooth equilibrium branch with a nonsingular and reasonably con-

Method	Theory (per optimizer update)		Task 1 (200 updates)		Task 2 (500 updates)		Task 3 (500 updates)	
	Time/update	Memory/update	Time (s)↓	Best loss↓	Time (s)↓	Best loss↓	Time (s)↓	Best loss↓
SPSA	$\mathcal{O}(2K C_{\text{eq}})$	$\mathcal{O}(n_{\Theta})$	727.3 ± 108.8	$2.0 \times 10^{-6} \pm 1.8 \times 10^{-6}$	3612.9 ± 1231.9	$5.4 \times 10^{-3} \pm 7.7 \times 10^{-3}$	3967.3 ± 417.9	$2.8 \times 10^{-5} \pm 1.5 \times 10^{-5}$
iCEM	$\mathcal{O}(PK C_{\text{eq}})$	$\mathcal{O}(P n_{\Theta})$	2332.2 ± 126.7	$1.5 \times 10^{-2} \pm 1.5 \times 10^{-2}$	3208.0 ± 13.7	$9.8 \times 10^{-2} \pm 3.6 \times 10^{-2}$	9907.3 ± 59.2	$1.2 \times 10^{-2} \pm 7.3 \times 10^{-3}$
Adjoint-only	$\mathcal{O}(K(C_{\text{eq}} + C_{\text{lin}}))$	$\mathcal{O}(K(n_x + n_z) + n_{\Theta})$	176.0 ± 53.1	$3.2 \times 10^{-7} \pm 5.6 \times 10^{-7}$	1111.2 ± 83.1	$4.3 \times 10^{-4} \pm 5.4 \times 10^{-4}$	935.2 ± 164.7	$8.4 \times 10^{-3} \pm 1.2 \times 10^{-2}$
Adjoint+RHC	$\mathcal{O}(H(C_{\text{eq}} + C_{\text{lin}}))$	$\mathcal{O}(H(n_x + n_z) + n_{\Theta})$	16.1 ± 2.8	$2.3 \times 10^{-7} \pm 3.0 \times 10^{-7}$	186.9 ± 24.8	$3.6 \times 10^{-8} \pm 3.6 \times 10^{-8}$	50.9 ± 6.1	$3.8 \times 10^{-8} \pm 7.4 \times 10^{-9}$

Table 1. Theory and empirical efficiency comparison. Here K is the number of continuation steps in a full-horizon rollout, H is the number of steps per RHC segment, P is the iCEM population size, and n_x, n_z, n_{Θ} denote the dimensions of $\mathbf{x}, \mathbf{z}$, and the neural-controller parameters Θ , respectively. C_{eq} is the cost of computing one converged equilibrium at a continuation step, and C_{lin} is the cost of one additional linearized tangent/adjoint solve with $\mathbf{G}_x$ or $\mathbf{G}_x^T$ used by implicit sensitivities. Memory for SPSA/iCEM counts optimizer-side storage beyond the working memory required by a single equilibrium solve. For adjoint methods, memory includes storing the realized trajectory states for the backward proxy-adjoint pass. For Adjoint+RHC, the listed theory cost is per optimizer update within one RHC segment; the reported wall-clock time sums all segment optimizations over the executed trajectory. The table reports total wall-clock runtime and best loss achieved over the fixed update budget.

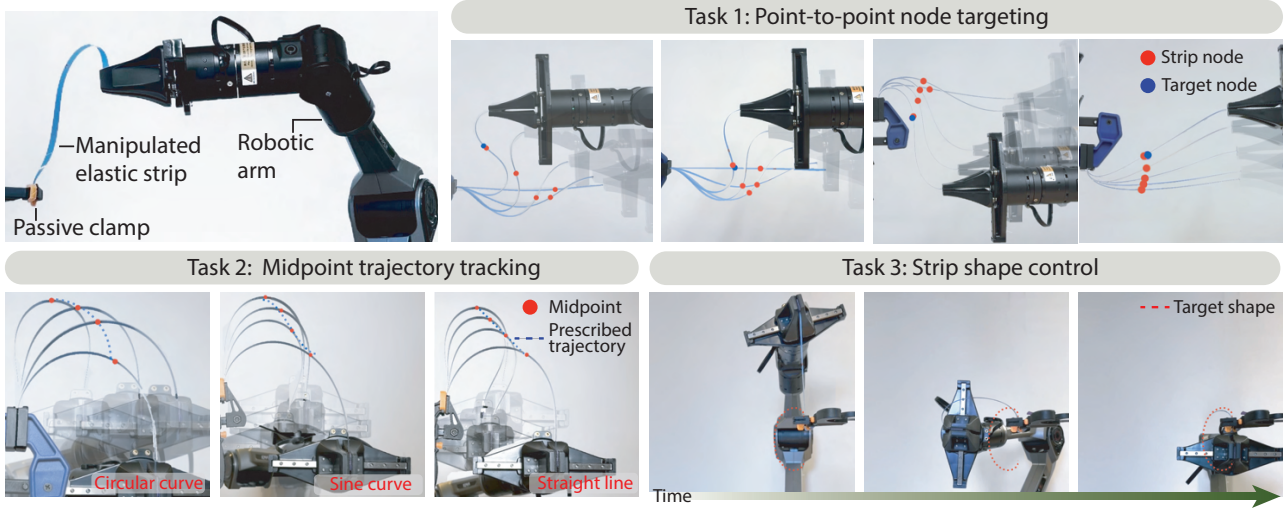


Figure 5. Experimental setup and qualitative results for all three tasks. For Task 1 and Task 2, each subfigure shows sampled frames from the control process, illustrating point-to-point manipulation and midpoint trajectory following, respectively. For Task 3, the figure visualizes the initial, intermediate, and final configurations during the shape control process.

ditioned $\mathbf{G}_x$; near bifurcations, singular equilibria, or high-curvature branches, the frozen-tangent approximation may become less accurate. In addition, performance depends on the quality of the differentiable implicit model, and unmodeled non-conservative effects such as friction, hysteresis, material damping, or intermittent hard contact can degrade transfer. Finally, equilibrium solves remain the dominant computational cost, so scaling to high-dimensional 2D/3D deformable bodies will require efficient solvers, preconditioning, warm starts, and adaptive continuation.

6. Conclusion

We presented Neural Control, a proxy-adjoint framework for learning and control through sequential implicit equilibria. The method keeps forward rollouts on converged equilibrium states while avoiding backpropagation through the inner Newton–CG iterations, and combines the resulting trajectory-dependent sensitivities with receding-horizon

continuation for robust branch tracking in multistable landscapes. Empirically, Adjoint+RHC achieves large efficiency gains across elastic-strip manipulation tasks: it is up to $195\times$ faster than the main SPSA/iCEM baselines while reaching substantially lower best loss under the same fixed update budgets. Hardware experiments further demonstrate qualitative sim-to-real transfer on a 6-DoF robotic setup under gravity and experimentally identified material parameters. Together, these results suggest that proxy-adjoint continuation is a practical gradient pathway for optimizing compliant, multistable systems whose states are defined by implicit equilibrium solves.

Acknowledgements

We acknowledge financial support from the National Institutes of Health (grant 1R01NS141171-01), the National Science Foundation (grant 2332555), and the National Science Foundation (grant 2332554).

Impact Statement

This work develops methods for learning and control in equilibrium-defined physical systems, with a focus on deformable object manipulation and compliant structures. Potential positive impacts include more efficient simulation-based control, improved robotic manipulation of flexible objects, and broader use of differentiable implicit models in physical AI. Potential risks arise if such controllers are deployed on hardware without adequate validation, especially under model mismatch, unmodeled dynamics, contact, or safety-critical constraints. We encourage deployment with appropriate hardware limits, task-specific validation, safety constraints, and human oversight.

References

- Agrawal, A., Amos, B., Barratt, S., Boyd, S., Diamond, S., and Kolter, J. Z. Differentiable convex optimization layers. *Advances in neural information processing systems*, 32, 2019.
- Amos, B. and Kolter, J. Z. Optnet: Differentiable optimization as a layer in neural networks. In *International conference on machine learning*, pp. 136–145. PMLR, 2017.
- Amos, B. and Yarats, D. The differentiable cross-entropy method. In *International Conference on Machine Learning*, pp. 291–302. PMLR, 2020.
- Amos, B., Jimenez, I., Sacks, J., Boots, B., and Kolter, J. Z. Differentiable MPC for end-to-end planning and control. In *Advances in Neural Information Processing Systems*, volume 31, 2018.
- Bai, S., Kolter, J. Z., and Koltun, V. Deep equilibrium models. *Advances in neural information processing systems*, 32, 2019.
- Bergou, M., Wardetzky, M., Robinson, S., Audoly, B., and Grinspun, E. Discrete elastic rods. In *ACM SIGGRAPH 2008 papers*, pp. 1–12. 2008.
- Bishop, C. M. Mixture density networks. 1994.
- Botev, Z. I., Kroese, D. P., Rubinstein, R. Y., and L’ecuyer, P. The cross-entropy method for optimization. In *Handbook of statistics*, volume 31, pp. 35–59. Elsevier, 2013.
- Bretl, T. and McCarthy, Z. Quasi-static manipulation of a kirchhoff elastic rod based on a geometric analysis of equilibrium configurations. *The International Journal of Robotics Research*, 33(1):48–68, 2014.
- Chen, R. T., Rubanova, Y., Bettencourt, J., and Duvenaud, D. K. Neural ordinary differential equations. *Advances in neural information processing systems*, 31, 2018.
- Chua, K., Calandra, R., McAllister, R., and Levine, S. Deep reinforcement learning in a handful of trials using probabilistic dynamics models. *Advances in neural information processing systems*, 31, 2018.
- Du, T., Wu, K., Ma, P., Wah, S., Spielberg, A., Rus, D., and Matusik, W. Diffpd: Differentiable projective dynamics. *ACM Transactions on Graphics (ToG)*, 41(2):1–21, 2021.
- Freeman, C. D., Frey, E., Raichuk, A., Girgin, S., Mordatch, I., and Bachem, O. Brax—a differentiable physics engine for large scale rigid body simulation. *arXiv preprint arXiv:2106.13281*, 2021.
- Gao, F., Shi, L., Zhang, S., Wang, Z., and Wu, Y. Adaptive-gradient policy optimization: Enhancing policy learning in non-smooth differentiable simulations. In *Forty-first International Conference on Machine Learning*, 2024.
- Georgiev, I., Srinivasan, K., Xu, J., Heiden, E., and Garg, A. Adaptive horizon actor-critic for policy learning in contact-rich differentiable simulation. *arXiv preprint arXiv:2405.17784*, 2024.
- Grinspun, E., Hirani, A. N., Desbrun, M., and Schröder, P. Discrete shells. In *Proceedings of the 2003 ACM SIGGRAPH/Eurographics Symposium on Computer Animation*, SCA ’03, pp. 62–67, Goslar, DEU, 2003. Eurographics Association. ISBN 1581136595.
- Gruslys, A., Munos, R., Danihelka, I., Lanctot, M., and Graves, A. Memory-efficient backpropagation through time. *Advances in neural information processing systems*, 29, 2016.
- Hafner, D., Lillicrap, T., Ba, J., and Norouzi, M. Dream to control: Learning behaviors by latent imagination. *arXiv preprint arXiv:1912.01603*, 2019a.
- Hafner, D., Lillicrap, T., Fischer, I., Villegas, R., Ha, D., Lee, H., and Davidson, J. Learning latent dynamics for planning from pixels. In *International conference on machine learning*, pp. 2555–2565. PMLR, 2019b.
- Hausknecht, M. J. and Stone, P. Deep recurrent q-learning for partially observable mdps. In *AAAI fall symposia*, volume 45, pp. 141, 2015.
- Hu, Y., Anderson, L., Li, T.-M., Sun, Q., Carr, N., Ragan-Kelley, J., and Durand, F. DiffTaichi: Differentiable programming for physical simulation. *arXiv preprint arXiv:1910.00935*, 2019a.
- Hu, Y., Liu, J., Spielberg, A., Tenenbaum, J. B., Freeman, W. T., Wu, J., Rus, D., and Matusik, W. Chainqueen: A real-time differentiable physical simulator for soft robotics. In *2019 International conference on robotics and automation (ICRA)*, pp. 6265–6271. IEEE, 2019b.

- Huang, W., Hao, Z., Li, J., Tong, D., Guo, K., Zhang, Y., Gao, H., Hsia, K. J., and Liu, M. A tutorial on simulating nonlinear behaviors of flexible structures with the discrete differential geometry (ddg) method. *Applied Mechanics Reviews*, pp. 1–88, 2025.
- Huang, Z., Hu, Y., Du, T., Zhou, S., Su, H., Tenenbaum, J. B., and Gan, C. Plasticinelab: A soft-body manipulation benchmark with differentiable physics. *arXiv preprint arXiv:2104.03311*, 2021.
- Jin, W., Wang, Z., Yang, Z., and Mou, S. Pontryagin differentiable programming: An end-to-end learning and control framework. *Advances in Neural Information Processing Systems*, 33:7979–7992, 2020.
- Li, S., Huang, Z., Chen, T., Du, T., Su, H., Tenenbaum, J. B., and Gan, C. Dexdeform: Dexterous deformable object manipulation with human demonstrations and differentiable physics. *arXiv preprint arXiv:2304.03223*, 2023.
- Lin, X., Wang, Y., Olkin, J., and Held, D. Softgym: Benchmarking deep reinforcement learning for deformable object manipulation. In *Conference on Robot Learning*, pp. 432–448. PMLR, 2021.
- Mania, H., Guy, A., and Recht, B. Simple random search of static linear policies is competitive for reinforcement learning. *Advances in neural information processing systems*, 31, 2018.
- Moll, M. and Kavraki, L. E. Path planning for deformable linear objects. *IEEE Transactions on Robotics*, 22(4): 625–636, 2006.
- Mora, M. A. Z., Peychev, M., Ha, S., Vechev, M., and Coros, S. Pods: Policy optimization via differentiable simulation. In *International Conference on Machine Learning*, pp. 7805–7817. PMLR, 2021.
- Pfaff, T., Fortunato, M., Sanchez-Gonzalez, A., and Battaglia, P. W. Learning mesh-based simulation with graph networks. In *9th International Conference on Learning Representations (ICLR)*. OpenReview.net, 2021. URL https://openreview.net/forum?id=roNqYL0_XP.
- Pinneri, C., Sawant, S., Blaes, S., Achterhold, J., Stueckler, J., Rolinek, M., and Martius, G. Sample-efficient cross-entropy method for real-time planning. In *Conference on Robot Learning*, pp. 1049–1065. PMLR, 2021.
- Qiao, Y., Liang, J., Koltun, V., and Lin, M. Differentiable simulation of soft multi-body systems. *Advances in Neural Information Processing Systems*, 34:17123–17135, 2021.
- Qiao, Y.-L., Liang, J., Koltun, V., and Lin, M. C. Scalable differentiable physics for learning and control. *arXiv preprint arXiv:2007.02168*, 2020.
- Salimans, T., Ho, J., Chen, X., Sidor, S., and Sutskever, I. Evolution strategies as a scalable alternative to reinforcement learning. *arXiv preprint arXiv:1703.03864*, 2017.
- Sanchez-Gonzalez, A., Godwin, J., Pfaff, T., Ying, R., Leskovec, J., and Battaglia, P. W. Learning to simulate complex physics with graph networks. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*, volume 119 of *Proceedings of Machine Learning Research*, pp. 8459–8468. PMLR, 2020. URL <http://proceedings.mlr.press/v119/sanchez-gonzalez20a.html>.
- Spall, J. C. Multivariate stochastic approximation using a simultaneous perturbation gradient approximation. *IEEE transactions on automatic control*, 37(3):332–341, 2002.
- Suh, H. J., Simchowitz, M., Zhang, K., and Tedrake, R. Do differentiable simulators give better policy gradients? In *International Conference on Machine Learning*, pp. 20668–20696. PMLR, 2022.
- Tong, D., Borum, A., and Jawed, M. K. Automated stability testing of elastic rods with helical centerlines using a robotic system. *IEEE Robotics and Automation Letters*, 7(2):1126–1133, 2021.
- Tong, D., Choi, A., Qin, L., Huang, W., Joo, J., and Jawed, M. K. Sim2real neural controllers for physics-based robotic deployment of deformable linear objects. *The International Journal of Robotics Research*, 43(6):791–810, 2024.
- Tong, D., Choi, A., Wang, J., Huang, W., Chen, Z., Li, J., Huang, X., Liu, M., Gao, H., and Hsia, K. J. Discrete differential geometry for simulating nonlinear behaviors of flexible systems: A survey. *Extreme Mechanics Letters*, pp. 102430, 2025.
- Williams, R. J. and Peng, J. An efficient gradient-based algorithm for on-line training of recurrent network trajectories. *Neural computation*, 2(4):490–501, 1990.
- Xing, E., Luk, V., and Oh, J. Stabilizing reinforcement learning in differentiable multiphysics simulation. In *The Thirteenth International Conference on Learning Representations (ICLR)*. OpenReview.net, 2025. URL <https://openreview.net/forum?id=DRiLWb8bJg>.
- Yin, H., Varava, A., and Kragic, D. Modeling, learning, perception, and control methods for deformable object manipulation. *Science Robotics*, 6(54):eabd8803, 2021.

A. Discrete Proxy-Adjoint Derivation and Implementation Details

This appendix derives the discrete proxy-adjoint recursions used in Section 3.1 and describes the implementation used to compute policy gradients without differentiating through the inner equilibrium solver. The key primitive is a matrix-free routine for products $\mathbf{S}_k^\top \mathbf{v}$, where $\mathbf{S}_k = \partial \mathbf{x}_k / \partial \mathbf{z}_k$ is the local equilibrium sensitivity at continuation step k .

Forward rollout and notation. We discretize $\lambda \in [0, 1]$ into $\{\lambda_k\}_{k=0}^K$ with step size $\Delta\lambda = 1/K$. At each step $k = 0, \dots, K-1$, the controller outputs

$$\mathbf{u}_k = \mathbf{u}_\Theta(\lambda_k), \quad (25)$$

and the continuation variable is updated explicitly as

$$\mathbf{z}_{k+1} = \mathbf{z}_k + \Delta\lambda f(\mathbf{z}_k, \mathbf{u}_k). \quad (26)$$

The state $\mathbf{x}_k$ is defined as the converged equilibrium at $\mathbf{z}_k$, warm-started from the previous solution:

$$\mathbf{x}_k = \text{SolveEq}(\mathbf{z}_k; \text{init} = \mathbf{x}_{k-1}), \quad \mathcal{G}(\mathbf{x}_k, \mathbf{z}_k) = \mathbf{0}. \quad (27)$$

The forward rollout always uses these converged equilibria. The proxy dynamics introduced below is used only for the backward sensitivity calculation.

Along the realized rollout, define the equilibrium Jacobians

$$\mathbf{G}_{x,k} := \left. \frac{\partial \mathcal{G}}{\partial \mathbf{x}} \right|_{(\mathbf{x}_k, \mathbf{z}_k, \lambda_k)}, \quad \mathbf{G}_{z,k} := \left. \frac{\partial \mathcal{G}}{\partial \mathbf{z}} \right|_{(\mathbf{x}_k, \mathbf{z}_k, \lambda_k)}, \quad (28)$$

and the continuation-dynamics Jacobians

$$\mathbf{A}_k := \left. \frac{\partial f}{\partial \mathbf{z}} \right|_{(\mathbf{z}_k, \mathbf{u}_k)}, \quad \mathbf{B}_k := \left. \frac{\partial f}{\partial \mathbf{u}} \right|_{(\mathbf{z}_k, \mathbf{u}_k)}. \quad (29)$$

Implicit sensitivity and $\mathbf{S}_k^\top \mathbf{v}$ products. Assuming that the realized branch is locally smooth and that $\mathbf{G}_{x,k}$ is nonsingular, implicit differentiation of $\mathcal{G}(\mathbf{x}_k, \mathbf{z}_k) = \mathbf{0}$ gives

$$\mathbf{S}_k := \frac{\partial \mathbf{x}_k}{\partial \mathbf{z}_k}, \quad \mathbf{G}_{x,k} \mathbf{S}_k = -\mathbf{G}_{z,k}. \quad (30)$$

We never form $\mathbf{S}_k$ explicitly. Instead, for any vector $\mathbf{v}$, we compute $\mathbf{S}_k^\top \mathbf{v}$ by solving the discrete adjoint linear system

$$\mathbf{G}_{x,k}^\top \boldsymbol{\eta}_k = \mathbf{v}, \quad (31)$$

and returning

$$\mathbf{S}_k^\top \mathbf{v} = -\mathbf{G}_{z,k}^\top \boldsymbol{\eta}_k. \quad (32)$$

This matrix-free product is the only place where equilibrium sensitivities enter the gradient computation.

Proxy dynamics and frozen tangent Since $\mathbf{x}_k$ is an implicit function of $\mathbf{z}_k$, the first-order branch-consistent variation satisfies

$$\delta \mathbf{x}_k \approx \mathbf{S}_k \delta \mathbf{z}_k. \quad (33)$$

Using the explicit continuation update (26), we define the discrete proxy transition

$$\mathbf{x}_{k+1} \approx \mathbf{x}_k + \Delta\lambda \mathbf{S}_k f(\mathbf{z}_k, \mathbf{u}_k). \quad (34)$$

Equation (34) is not used to generate the forward trajectory; the forward pass still solves Eq. (27) to convergence. Instead, Eq. (34) defines a tractable backward pass.

The frozen-tangent approximation treats $\mathbf{S}_k$ as locally constant during the backward pass. Equivalently, we ignore derivatives of $\mathbf{S}_k$ with respect to $\mathbf{x}_k$, $\mathbf{z}_k$, and λ_k . This avoids higher-order derivatives of the equilibrium residual. Receding-horizon continuation limits the accumulation of this proxy mismatch by restricting the backward propagation horizon (Section 3.3).

Unified discrete objective. We write terminal and trajectory objectives in the form

$$\mathcal{L} = \phi(\mathbf{x}_K, \mathbf{z}_K) + \sum_{k=0}^{K-1} w_k \ell(\mathbf{x}_k, \mathbf{z}_k), \quad (35)$$

where ϕ is an optional terminal loss and ℓ is an optional running loss. Terminal-only objectives use $w_k \equiv 0$; trajectory objectives set $\phi \equiv 0$ if no terminal loss is needed. The scalar w_k includes the quadrature weight; for a Riemann approximation of $\int_0^1 w(\lambda) \ell d\lambda$, one may take $w_k = \Delta\lambda w(\lambda_k)$.

Proxy adjoints. Define proxy-system adjoints

$$\mathbf{a}_k := \frac{\partial \mathcal{L}}{\partial \mathbf{x}_k}, \quad \mathbf{g}_k := \frac{\partial \mathcal{L}}{\partial \mathbf{z}_k}. \quad (36)$$

The terminal conditions are

$$\mathbf{a}_K = \nabla_{\mathbf{x}} \phi(\mathbf{x}_K, \mathbf{z}_K), \quad \mathbf{g}_K = \nabla_{\mathbf{z}} \phi(\mathbf{x}_K, \mathbf{z}_K). \quad (37)$$

The dependence of the equilibrium state on the continuation variable is accounted for through $\mathbf{S}_k^\top \mathbf{a}_{k+1}$ in the backward recursion and control gradient below.

Under the proxy transition (34) and frozen tangent, the linearized transitions are

$$\frac{\partial \mathbf{z}_{k+1}}{\partial \mathbf{z}_k} = \mathbf{I} + \Delta\lambda \mathbf{A}_k, \quad \frac{\partial \mathbf{z}_{k+1}}{\partial \mathbf{u}_k} = \Delta\lambda \mathbf{B}_k, \quad (38)$$

and

$$\frac{\partial \mathbf{x}_{k+1}}{\partial \mathbf{x}_k} = \mathbf{I}, \quad \frac{\partial \mathbf{x}_{k+1}}{\partial \mathbf{z}_k} \approx \Delta\lambda \mathbf{S}_k \mathbf{A}_k, \quad \frac{\partial \mathbf{x}_{k+1}}{\partial \mathbf{u}_k} \approx \Delta\lambda \mathbf{S}_k \mathbf{B}_k. \quad (39)$$

Therefore, for $k = K-1, \dots, 0$, the adjoints satisfy

$$\mathbf{a}_k \approx \mathbf{a}_{k+1} + w_k \nabla_{\mathbf{x}} \ell(\mathbf{x}_k, \mathbf{z}_k), \quad (40)$$

and

$$\begin{aligned} \mathbf{g}_k &\approx (\mathbf{I} + \Delta\lambda \mathbf{A}_k)^\top \mathbf{g}_{k+1} + \Delta\lambda \mathbf{A}_k^\top (\mathbf{S}_k^\top \mathbf{a}_{k+1}) \\ &\quad + w_k \nabla_{\mathbf{z}} \ell(\mathbf{x}_k, \mathbf{z}_k). \end{aligned} \quad (41)$$

Each product $\mathbf{S}_k^\top (\cdot)$ is evaluated using Eq. (32). Notice that $\mathbf{S}_k^\top \nabla_{\mathbf{x}} \ell$ does not appear separately in Eq. (41); the running-state loss contribution is first accumulated in $\mathbf{a}_k$ through Eq. (40), and then converted to a $\mathbf{z}$ -space contribution through $\mathbf{S}_k^\top \mathbf{a}$.

For convenience define the effective continuation adjoint

$$\mathbf{d}_k := \mathbf{g}_{k+1} + \mathbf{S}_k^\top \mathbf{a}_{k+1}. \quad (42)$$

The control gradient at each step is

$$\frac{d\mathcal{L}}{d\mathbf{u}_k} \approx \Delta\lambda \mathbf{B}_k^\top \mathbf{d}_k. \quad (43)$$

The policy-parameter gradient then follows by the chain rule through the controller:

$$\frac{d\mathcal{L}}{d\Theta} = \sum_{k=0}^{K-1} \left(\frac{\partial \mathbf{u}_\Theta(\lambda_k)}{\partial \Theta} \right)^\top \frac{d\mathcal{L}}{d\mathbf{u}_k}. \quad (44)$$

Implementation. Algorithm 1 summarizes the implementation of proxy-adjoint training with receding-horizon continuation.

B. Elastic Strip Equilibrium Model and Implicit Solver

This section instantiates the equilibrium residual $\mathcal{G}(\mathbf{x}, \mathbf{z})$ and the implicit solver $\mathbf{x}_k = \text{SolveEq}(\mathbf{z}_k)$ used in the elastic-strip experiments.

Algorithm 1 Adjoint+RHC: proxy-adjoint training with receding-horizon continuation

Input: segment horizon H , number of segments M , step size $\Delta\lambda$
Input: initial $(\mathbf{x}^{(0)}, \mathbf{z}^{(0)})$, controller $\mathbf{u}_\Theta(\lambda)$, continuation dynamics $f(\mathbf{z}, \mathbf{u})$
Input: segment update budget T_{seg} , optimizer step $\text{OptStep}(\Theta, d\mathcal{L}/d\Theta)$
Helper: $\text{StProd}(k, \mathbf{v})$ computes $\mathbf{S}_k^\top \mathbf{v}$:
 solve $\mathbf{G}_{x,k}^\top \boldsymbol{\eta}_k = \mathbf{v}$, return $-\mathbf{G}_{z,k}^\top \boldsymbol{\eta}_k$

for $m = 0$ **to** $M - 1$ **do**
 Set segment start $(\mathbf{x}_0, \mathbf{z}_0) \leftarrow (\mathbf{x}^{(m)}, \mathbf{z}^{(m)})$
 Initialize $\Theta \leftarrow \Theta_0$
 for $t = 1$ **to** T_{seg} **do**
 Forward rollout with converged equilibria.
 for $k = 0$ **to** $H - 1$ **do**
 $\lambda_k^{(m)} \leftarrow (mH + k)\Delta\lambda$
 $\mathbf{u}_k \leftarrow \mathbf{u}_\Theta(\lambda_k^{(m)})$
 Cache $\mathbf{G}_{x,k}, \mathbf{G}_{z,k}, \mathbf{A}_k, \mathbf{B}_k$ at $(\mathbf{x}_k, \mathbf{z}_k, \mathbf{u}_k)$
 $\mathbf{z}_{k+1} \leftarrow \mathbf{z}_k + \Delta\lambda f(\mathbf{z}_k, \mathbf{u}_k)$
 $\mathbf{x}_{k+1} \leftarrow \text{SolveEq}(\mathbf{z}_{k+1}; \text{init} = \mathbf{x}_k)$
 end for
 Compute segment loss $\mathcal{L}$ on the realized segment rollout
 Backward pass with proxy adjoints and frozen tangent.
 Initialize $\mathbf{a}_H \leftarrow \partial\mathcal{L}/\partial\mathbf{x}_H$
 Initialize $\mathbf{g}_H \leftarrow \partial\mathcal{L}/\partial\mathbf{z}_H$
 for $k = H - 1$ **to** 0 **do**
 $\mathbf{s}_k \leftarrow \text{StProd}(k, \mathbf{a}_{k+1})$
 $\mathbf{d}_k \leftarrow \mathbf{g}_{k+1} + \mathbf{s}_k$
 $d\mathcal{L}/d\mathbf{u}_k \leftarrow \Delta\lambda \mathbf{B}_k^\top \mathbf{d}_k$
 $\mathbf{a}_k \leftarrow \mathbf{a}_{k+1} + w_k \nabla_{\mathbf{x}} \ell(\mathbf{x}_k, \mathbf{z}_k)$
 $\mathbf{g}_k \leftarrow (\mathbf{I} + \Delta\lambda \mathbf{A}_k)^\top \mathbf{g}_{k+1} + \Delta\lambda \mathbf{A}_k^\top \mathbf{s}_k + w_k \nabla_{\mathbf{z}} \ell(\mathbf{x}_k, \mathbf{z}_k)$
 end for
 $d\mathcal{L}/d\Theta \leftarrow \sum_{k=0}^{H-1} \left(\partial\mathbf{u}_\Theta(\lambda_k^{(m)})/\partial\Theta \right)^\top (d\mathcal{L}/d\mathbf{u}_k)$
 $\Theta \leftarrow \text{OptStep}(\Theta, d\mathcal{L}/d\Theta)$
 end for
 Execute segment once with optimized Θ .
 Roll out from $(\mathbf{x}^{(m)}, \mathbf{z}^{(m)})$ with optimized Θ to obtain terminal $(\mathbf{x}_H, \mathbf{z}_H)$
 Set $(\mathbf{x}^{(m+1)}, \mathbf{z}^{(m+1)}) \leftarrow (\mathbf{x}_H, \mathbf{z}_H)$
end for
Output: concatenated executed rollout and task loss evaluated over all segments

State and controls. As shown in Fig. 6, we discretize the strip centerline into $N+1$ nodes $\{\mathbf{q}_i\}_{i=0}^N$ with $\mathbf{q}_i \in \mathbb{R}^d$ ($d=2$ or 3), and stack them as $\mathbf{q} = [\mathbf{q}_0^\top, \dots, \mathbf{q}_N^\top]^\top$. We clamp two nodes at each end and treat them as boundary (controllable) DoFs,

$$\begin{aligned}
 \mathbf{q}_b &:= [\mathbf{q}_0^\top, \mathbf{q}_1^\top, \mathbf{q}_{N-1}^\top, \mathbf{q}_N^\top]^\top, \\
 \mathbf{q}_f &:= [\mathbf{q}_2^\top, \dots, \mathbf{q}_{N-2}^\top]^\top.
 \end{aligned} \tag{45}$$

We identify $\mathbf{x} \equiv \mathbf{q}_f$ and $\mathbf{z} \equiv \mathbf{q}_b$.

Energy model. Strip equilibrium is defined by a stationary point of the total potential energy

$$E_{\text{tot}}(\mathbf{q}_f) = E_s(\mathbf{q}_f, \mathbf{q}_b) + E_b(\mathbf{q}_f, \mathbf{q}_b) + E_g(\mathbf{q}_f, \mathbf{q}_b), \tag{46}$$

including stretching, bending, and gravity.²

²We do not model twist; the strip is represented by its centerline with stretching and bending energies.

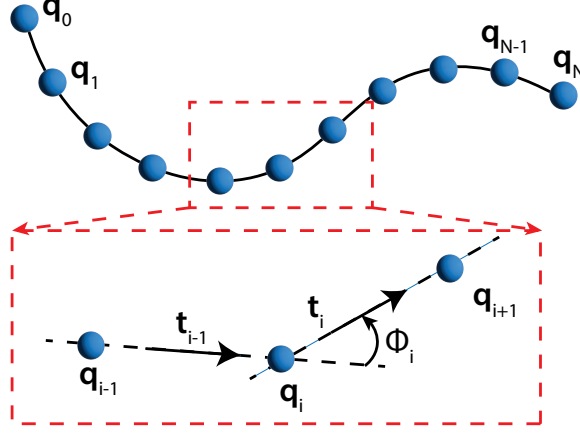


Figure 6. Discrete schematic of the elastic strip. The centerline is represented by nodal positions $\{\mathbf{q}_i\}$, edge vectors $\{\mathbf{e}_i\}$, unit tangents $\{\mathbf{t}_i\}$, and turning angles $\{\phi_i\}$.

Stretching energy. Let Δl be the undeformed edge length and $\mathbf{e}_i := \mathbf{q}_{i+1} - \mathbf{q}_i$ for $i = 0, \dots, N-1$. We use

$$E_s = \frac{1}{2} k_s \sum_{i=0}^{N-1} \left(1 - \frac{\|\mathbf{e}_i\|}{\Delta l} \right)^2 \Delta l. \quad (47)$$

Bending energy. Let ϕ_i be the turning angle at node i (Fig. 6) and ϕ_i^0 the rest angle (zero in our study). We use

$$E_b = \frac{1}{2} k_b \sum_{i=1}^{N-1} \left(2 \tan \frac{\phi_i}{2} - 2 \tan \frac{\phi_i^0}{2} \right)^2 \frac{1}{\Delta l}. \quad (48)$$

Gravitational energy. We use a lumped-mass gravity potential

$$E_g = - \sum_{i=0}^N m_i \mathbf{g}^\top \mathbf{q}_i, \quad (49)$$

where $\mathbf{g} \in \mathbb{R}^d$ is the gravitational acceleration and m_i is the lumped nodal mass.

Equilibrium residual. Given $\mathbf{q}_b$, the free configuration returned by SolveEq is a locally stable equilibrium selected by the warm-started solver. Equivalently, it satisfies the stationarity condition

$$\nabla_{\mathbf{q}_f} E_{\text{tot}}(\mathbf{q}_f^*, \mathbf{q}_b) = \mathbf{0}, \quad (50)$$

with the particular local branch determined by the previous continuation step. We therefore define

$$\begin{aligned} \mathcal{G}(\mathbf{x}, \mathbf{z}) &:= \nabla_{\mathbf{q}_f} E_{\text{tot}}(\mathbf{q}_f, \mathbf{q}_b), \\ \text{with } \mathbf{x} &\equiv \mathbf{q}_f, \quad \mathbf{z} \equiv \mathbf{q}_b. \end{aligned} \quad (51)$$

The Jacobians $\mathbf{G}_{x,k}$ and $\mathbf{G}_{z,k}$ in Eq. (28) are the corresponding derivatives of $\mathcal{G}$ evaluated at $(\mathbf{q}_{f,k}, \mathbf{q}_{b,k})$; namely, $\mathbf{G}_{x,k} = \nabla_{\mathbf{q}_f}^2 E_{\text{tot}}$ and $\mathbf{G}_{z,k} = \nabla_{\mathbf{q}_f \mathbf{q}_b}^2 E_{\text{tot}}$.

Implicit solver (SolveEq). We implement $\mathbf{x}_k = \text{SolveEq}(\mathbf{z}_k, \lambda_k)$ using a trust-region Newton–CG method applied to the stationarity condition (50). At iterate $\mathbf{q}_f^{(t)}$, we assemble $E_{\text{tot}}^{(t)}$, $\mathbf{g}^{(t)} := \nabla_{\mathbf{q}_f} E_{\text{tot}}$, and $\mathbf{H}^{(t)} := \nabla_{\mathbf{q}_f}^2 E_{\text{tot}}$, and symmetrize $\mathbf{H}^{(t)} \leftarrow \frac{1}{2}(\mathbf{H}^{(t)} + \mathbf{H}^{(t)\top})$.

Trust-region step. We approximately solve

$$\min_{\|\mathbf{p}\|_2 \leq \Delta^{(t)}} m^{(t)}(\mathbf{p}) := \mathbf{g}^{(t)\top} \mathbf{p} + \frac{1}{2} \mathbf{p}^\top \mathbf{H}^{(t)} \mathbf{p}. \quad (52)$$

If $\mathbf{H}^{(t)}$ is SPD (via Cholesky-type factorization), we compute the Newton step $\mathbf{H}^{(t)} \mathbf{p}_N^{(t)} = -\mathbf{g}^{(t)}$ and take $\mathbf{p}^{(t)} = \mathbf{p}_N^{(t)}$ if feasible; otherwise we take a standard dogleg boundary step between the Cauchy and Newton directions. If SPD detection fails, we use Steihaug PCG with Hessian-vector products, terminating on negative curvature $\mathbf{d}^\top \mathbf{H}^{(t)} \mathbf{d} \leq 0$ or on hitting $\|\mathbf{p}\|_2 = \Delta^{(t)}$. We use a Jacobi preconditioner $\mathbf{M}^{(t)} = \text{diag}(|\text{diag}(\mathbf{H}^{(t)})|) + \sigma \mathbf{I}$.

Acceptance, radius update, and termination. Let $\text{pred}^{(t)} = -m^{(t)}(\mathbf{p}^{(t)})$ and $\text{ared}^{(t)} = E^{(t)} - E_{\text{tot}}(\mathbf{q}_f^{(t)} + \mathbf{p}^{(t)}, \mathbf{q}_b; \lambda)$, and define $\rho^{(t)} = \text{ared}^{(t)} / \max(\text{pred}^{(t)}, \epsilon)$. We accept the step if $\rho^{(t)} \geq \eta$, and update $\Delta^{(t)}$ using standard shrink/expand rules based on $\rho^{(t)}$. We terminate when $\|\mathbf{g}^{(t)}\|_2 \leq \varepsilon_g$, $\Delta^{(t)} \leq \varepsilon_\Delta$, or after a maximum number of iterations. We warm-start continuation step k with the previous equilibrium $\mathbf{q}_{f,k}^{(0)} \leftarrow \mathbf{q}_{f,k-1}^*$; under multistability, this initialization selects a specific locally stable equilibrium branch for the same $\mathbf{z}$.

C. Assumptions and Consistency of the Frozen-Tangent Proxy Gradient

This section summarizes the assumptions under which the frozen-tangent proxy gradient used in Sec. 3.1 provides a controlled local approximation to the gradient along the realized equilibrium branch.

We consider the equilibrium constraint

$$\mathcal{G}(\mathbf{x}(\lambda), \mathbf{z}(\lambda)) = \mathbf{0}, \quad \lambda \in [0, 1], \quad (53)$$

where $\mathbf{z}(\lambda)$ is generated by the continuation dynamics $d\mathbf{z}/d\lambda = f(\mathbf{z}(\lambda), \mathbf{u}(\lambda))$. Let

$$\mathbf{G}_x = \frac{\partial \mathcal{G}}{\partial \mathbf{x}}, \quad \mathbf{G}_z = \frac{\partial \mathcal{G}}{\partial \mathbf{z}}, \quad \mathbf{G}_\lambda = \frac{\partial \mathcal{G}}{\partial \lambda}.$$

Assumption C.1 (Smooth realized branch). The residual $\mathcal{G}$ is C^2 in a neighborhood of the realized rollout, and the warm-started solver follows a locally unique equilibrium branch $\mathbf{x}(\lambda) \in C^1([0, 1])$. Along this branch, $\mathbf{G}_x(\lambda)$ is uniformly nonsingular:

$$\sup_{\lambda \in [0, 1]} \|\mathbf{G}_x(\lambda)^{-1}\| < \infty. \quad (54)$$

This assumption excludes singular equilibria, loss of local stability, and basin switches, where $\mathbf{G}_x$ may become ill-conditioned or singular.

Implicit sensitivity. Under Assumption C.1, the implicit function theorem gives a local equilibrium map $\mathbf{x}^*(\mathbf{z}, \lambda)$. Its sensitivity with respect to $\mathbf{z}$ is

$$\mathbf{S}(\lambda) := \frac{\partial \mathbf{x}^*}{\partial \mathbf{z}}(\lambda), \quad \mathbf{G}_x(\lambda) \mathbf{S}(\lambda) = -\mathbf{G}_z(\lambda). \quad (55)$$

Differentiating Eq. (53) along the realized branch gives

$$\frac{d\mathbf{x}}{d\lambda} = \mathbf{S}(\lambda) \frac{d\mathbf{z}}{d\lambda} - \mathbf{G}_x(\lambda)^{-1} \mathbf{G}_\lambda(\lambda). \quad (56)$$

In our experiments, explicit λ -dependence is either absent or absorbed into $\mathbf{z}(\lambda)$, so $\mathbf{G}_\lambda \equiv \mathbf{0}$ and

$$\frac{d\mathbf{x}}{d\lambda} = \mathbf{S}(\lambda) f(\mathbf{z}(\lambda), \mathbf{u}(\lambda)). \quad (57)$$

Frozen-tangent approximation. On a discretization $0 = \lambda_0 < \lambda_1 < \dots < \lambda_K = 1$, the backward pass evaluates $\mathbf{S}_k := \mathbf{S}(\lambda_k)$ on the realized rollout and treats it as constant over the interval $[\lambda_k, \lambda_{k+1}]$. The local tangent variation on this interval is

$$\eta_k := \sup_{\lambda \in [\lambda_k, \lambda_{k+1}]} \|\mathbf{S}(\lambda) - \mathbf{S}_k\|. \quad (58)$$

Let

$$\eta_S := \max_k \eta_k.$$

If $\mathbf{S}$ is Lipschitz continuous with constant L_S , then

$$\eta_S \leq L_S \Delta\lambda, \quad \Delta\lambda = \max_k (\lambda_{k+1} - \lambda_k). \quad (59)$$

Proposition C.2 (Local consistency of the frozen-tangent gradient). *Consider the trajectory objective in Eq. (16) over a fixed continuation segment. Let $\nabla_{\Theta} \mathcal{L}$ denote the branch-consistent gradient obtained by differentiating along the realized smooth equilibrium branch, and let $\nabla_{\Theta} \mathcal{L}_{\text{proxy}}$ denote the gradient obtained by freezing $\mathbf{S}(\lambda)$ on each continuation interval. Suppose Assumption C.1 holds, $\mathbf{S}(\lambda)$ has bounded local variation on the segment, and f , its first derivatives, the controller Jacobian $\partial \mathbf{u}_{\Theta} / \partial \Theta$, and the objective derivatives are uniformly bounded along the realized rollout. Then*

$$\|\nabla_{\Theta} \mathcal{L}_{\text{proxy}} - \nabla_{\Theta} \mathcal{L}\| \leq C \eta_S, \quad (60)$$

where C depends on the above bounds and the segment length, but not on η_S . In particular, if $\mathbf{S}$ is Lipschitz continuous, then $\eta_S = O(\Delta\lambda)$ and the proxy-gradient error is $O(\Delta\lambda)$ on a fixed segment.

Proof. Under Assumption C.1, the implicit function theorem gives a locally smooth equilibrium map and bounded local sensitivities. The exact linearized branch dynamics uses the continuously varying tangent $\mathbf{S}(\lambda)$, whereas the proxy backward pass replaces it on each interval by the frozen value $\mathbf{S}_k$. The resulting perturbation in the linearized dynamics is bounded by η_k on interval k . Standard stability estimates for linear variational and adjoint equations then bound the corresponding costate error by a constant times η_S over a fixed segment. Multiplying by the bounded controller Jacobian and summing over the segment yields Eq. (60). $\square$

The constant C may grow with the segment length, which motivates the receding-horizon design used in Neural Control.

Role of receding-horizon continuation. RHC reduces the accumulation of frozen-tangent error by applying the proxy gradient over short segments and re-anchoring each segment at the realized equilibrium. This does not remove the underlying ill-conditioning near bifurcations or basin switches, but it limits the horizon over which inaccurate local tangents can accumulate. When $\mathbf{G}_x$ becomes singular, when the solver switches basins, or when $\mathbf{S}(\lambda)$ varies rapidly, the bound above may no longer hold; these cases correspond to the primary failure modes of the frozen-tangent approximation.

D. Implementation Details and Hyperparameters

This section reports the hyperparameters used in our numerical experiments (Secs. 4.3–4.4). Unless otherwise stated, all methods share the same forward equilibrium simulator and the same controller parameterization; they differ only in the optimizer.

Elastic-strip simulator and equilibrium solver. We discretize the strip centerline into $N+1 = 101$ nodes and clamp the first two and last two nodes. Unless otherwise stated, material parameters are $k_b = 7.85 \times 10^{-8} \text{ N} \cdot \text{m}^2$ and $k_s = 0.314 \text{ N}$, and we set $\mathbf{g} = \mathbf{0}$ (no gravity) in all numerical experiments.

At each continuation step we solve for the free DoFs using a trust-region Newton–CG method (App. B), warm-started from the previous equilibrium. We use initial radius $\Delta_0 = 10^{-3}$, acceptance threshold $\eta = 0.1$, shrink/expand factors (0.5, 2.0), and at most 200 outer iterations. We terminate when $\|\nabla E_{\text{tot}}\| \leq 10^{-9}$ (gradient w.r.t. free DoFs) or when the radius drops below 10^{-8} .

If the Hessian is detected SPD we take a Newton/dogleg step; otherwise we use Steihaug PCG in the trust region. PCG is capped at 200 iterations and stops when the PCG residual satisfies $\|r\| \leq \min(0.5, \sqrt{\|\nabla E\|}) \|\nabla E\|$, on negative curvature, or upon hitting the trust-region boundary. We use a diagonal (Jacobi) preconditioner.

Controller parameterization and adjoint optimization. We parameterize the continuation controller $u_{\Theta}(\lambda)$ as a lightweight MLP that takes the scalar $\lambda \in [0, 1]$ as input and outputs the control vector $u(\lambda) \in \mathbb{R}^{d_u}$. For Tasks 1–2, $d_u = 2$ (left/right transverse displacement rates); for Task 3, $d_u = 3$ (end-effector (x, y) displacement rates and in-plane

rotation rate). Unless otherwise stated, we use 2–3 hidden layers with width 64 (architectures $[64, 64]$ or $[64, 64, 64]$), ReLU activations, and a final tanh to bound the output; we scale the output to enforce $\|u(\lambda)\| \leq u_{\max}$.

For the adjoint-only baseline, we use a single rollout with full horizon $K = 101$ continuation steps. For Adjoint+RHC, we use segment horizon $H = 10$ continuation steps per segment (with horizon shifting as described in Sec. 3.3). Both adjoint-based methods use Adam with learning rate 10^{-2} to optimize Θ .

SPSA baseline. SPSA is used as a black-box optimizer over the same controller parameters Θ . We use a two-sided perturbation estimator with perturbation magnitude $c = 5 \times 10^{-3}$ and learning rate 10^{-2} . Each SPSA update uses one perturbation pair, corresponding to two objective evaluations. We choose a relatively large c to obtain a stable finite-difference directional signal under strongly nonlinear deformation; with unit-normalized strip length, $c = 5 \times 10^{-3}$ corresponds to a small ($\sim 0.5\%$) geometric perturbation that empirically yields more reliable search directions than smaller perturbations that can be dominated by solver tolerance and numerical noise.

iCEM baseline. iCEM is used as a black-box optimizer over the same controller parameters Θ . At each update, iCEM samples a population of candidate controller parameters, evaluates each candidate by a full equilibrium-defined rollout, selects elite candidates, smooths the population update, and reinserts the current best candidate. We use population size $P = 10$, elite fraction 0.3, smoothing coefficient $\alpha = 0.25$, minimum standard deviation 10^{-2} , elite reuse fraction 1.0, no population decay, and best-candidate reinsertion. When the initial standard deviation is not specified, it is set to $0.5u_{\max}$. We use temporally correlated perturbations with correlation parameter $\beta = 2.0$ in the controller-induced actuation schedule.

Update budgets and reporting. Task 1 uses 200 optimizer updates, while Tasks 2–3 use 500 optimizer updates. For RHC, an update refers to one optimizer update within the current segment, and reported task losses are computed on the concatenated executed trajectory across segments. Wall-clock time in Table 1 includes all equilibrium solves and the additional tangent/adjoint linear solves used by implicit sensitivities. CEM hyperparameters and results are reported separately in Appendix E.

E. Additional CEM Baseline Results

In the main text, we report SPSA and iCEM as the primary gradient-free baselines. Here we additionally report results for the classical cross-entropy method (CEM), which was used in the original submission. CEM is applied as a black-box optimizer over the same controller parameters Θ and is run without RHC. At each update, CEM samples a population of candidate controller parameters, evaluates each candidate using a full equilibrium-defined rollout, selects elite candidates, and updates the search distribution. We use population size $P = 10$ and elite fraction $\rho = 0.3$.

Method	Task 1 (200 updates)		Task 2 (500 updates)		Task 3 (500 updates)	
	Time (s)↓	Best loss↓	Time (s)↓	Best loss↓	Time (s)↓	Best loss↓
CEM	2043.2 ± 73.0	$6.7 \times 10^{-4} \pm 1.2 \times 10^{-3}$	5358.9 ± 208.7	$6.4 \times 10^{-3} \pm 3.9 \times 10^{-3}$	3176.4 ± 232.8	$1.6 \times 10^{-2} \pm 8.5 \times 10^{-3}$
Adjoint+RHC	16.1 ± 2.8	$2.3 \times 10^{-7} \pm 3.0 \times 10^{-7}$	186.9 ± 24.8	$3.6 \times 10^{-8} \pm 3.6 \times 10^{-8}$	50.9 ± 6.1	$3.8 \times 10^{-8} \pm 7.4 \times 10^{-9}$

Table 2. Additional CEM baseline. CEM is evaluated under the same fixed update budgets as the main experiments and uses the same controller parameterization $\mathbf{u}_{\Theta}(\lambda)$. Each CEM candidate requires a full equilibrium-defined rollout. The results are consistent with the main comparison: Adjoint+RHC achieves substantially lower loss with much less wall-clock time.

F. Learned DEQ-Style Implicit Equilibrium Model

To test whether Neural Control applies beyond analytical mechanics models, we evaluate it on a learned DEQ-style implicit equilibrium model trained from experimental force–strain data. We collect force–strain measurements from a slinky and train a neural energy model whose equilibrium response is defined implicitly. After training, the model is frozen and used as the forward implicit model for Neural Control. We then optimize a force trajectory so that the equilibrium strain tracks a prescribed sinusoidal reference. Local sensitivities are computed by implicit differentiation of the learned equilibrium residual, using the same proxy-adjoint pipeline as in the elastic-strip experiments.

The learned implicit model closely fits the experimental force–strain response, and Neural Control successfully optimizes the input force trajectory to track the target strain. The resulting tracking losses over the optimized segments are on the order of 10^{-8} – 10^{-11} . This validation demonstrates that the proposed proxy-adjoint continuation framework is not tied to analytical elastic-strip simulators; it can also operate on learned implicit equilibrium models, provided that local implicit

sensitivities are available. Additional plots, including the force–strain fit, training curve, and tracking result, are provided on the project page¹.

G. System Identification for the Elastic Strip in Real Robotic Manipulation

This section describes the lightweight system-identification procedure used to match the elastic-strip model to the real-robot experiments. The key identified quantity is the gravito-bending length L_{gb} , which captures the characteristic deformation scale of a slender strip under gravity and is robust to many nuisance factors, such as camera noise and slight boundary mismatch.

Gravito-bending length. For an inextensible/slender strip under its own weight, the dominant competition is between bending stiffness and distributed gravity load. We parameterize this balance by

$$L_{gb} := \left(\frac{k_b}{\rho w h g} \right)^{\frac{1}{3}}, \quad (61)$$

where k_b is the bending stiffness (units $\text{N}\cdot\text{m}^2$), ρ is the density, w is the strip width, h is the thickness, and g is gravitational acceleration.

We collect $M = 5$ static snapshots of the strip under gravity with diverse boundary conditions (multiple clamp-clamp configurations). Each snapshot provides an observed equilibrium shape $\hat{x}^{(m)}$ together with the corresponding boundary constraints. Using multiple configurations reduces degeneracy and discourages overfitting to a single pose.

For a candidate L_{gb} , we run the simulator under each experimental boundary condition and obtain the converged equilibrium shape $x^{(m)}(L_{gb})$. We measure mismatch on *free nodes only* (excluding constrained/clamped DoFs) and use a mean-squared shape discrepancy:

$$J(L_{gb}) = \frac{1}{M} \sum_{m=1}^M \frac{1}{|\mathcal{I}_f^{(m)}|} \sum_{i \in \mathcal{I}_f^{(m)}} \left\| x_i^{(m)}(L_{gb}) - \hat{x}_i^{(m)} \right\|_2^2, \quad (62)$$

where $\mathcal{I}_f^{(m)}$ denotes the free-node index set for snapshot m . We identify

$$L_{gb}^* \in \arg \min_{L_{gb} \in [L_{\min}, L_{\max}]} J(L_{gb}). \quad (63)$$

We solve Eq. (63) with a bounded one-dimensional scalar minimization (Brent-type method). Unless otherwise stated, we use bounds $[L_{\min}, L_{\max}] = [0.05, 0.10]$ m and tolerance $\text{xtol} = 10^{-4}$. Each objective evaluation runs the forward simulator with gravity enabled and otherwise matches the real-robot settings. The identified value is $L_{gb} = 0.0514$ m.